USER INTERFACE WALKTHROUGH

The application, screen by screen

Real screenshots of the running app: what each screen shows and which rule it proves.

Sign-in | Invoices | Approvals | Refusals | Evidence

Screen-by-screen edition · Version 1.0 · 30 October 2026

BEFORE YOU START

Four accounts, three roles

Account	Role	Can approve	Also can
demo.manager	Manager	Invoices up to 50,000.00	Create, submit, reject
demo.director	Director	Up to 500,000.00 alone; above that as first or second approver	Create, submit, reject
demo.director2	Director	The same as demo.director: a second, different person	Create, submit, reject
demo.admin	Admin	Nothing: Admin never approves	Create, submit, reject, delete

Password for all four: `ChangeMe123!` (local demo only).

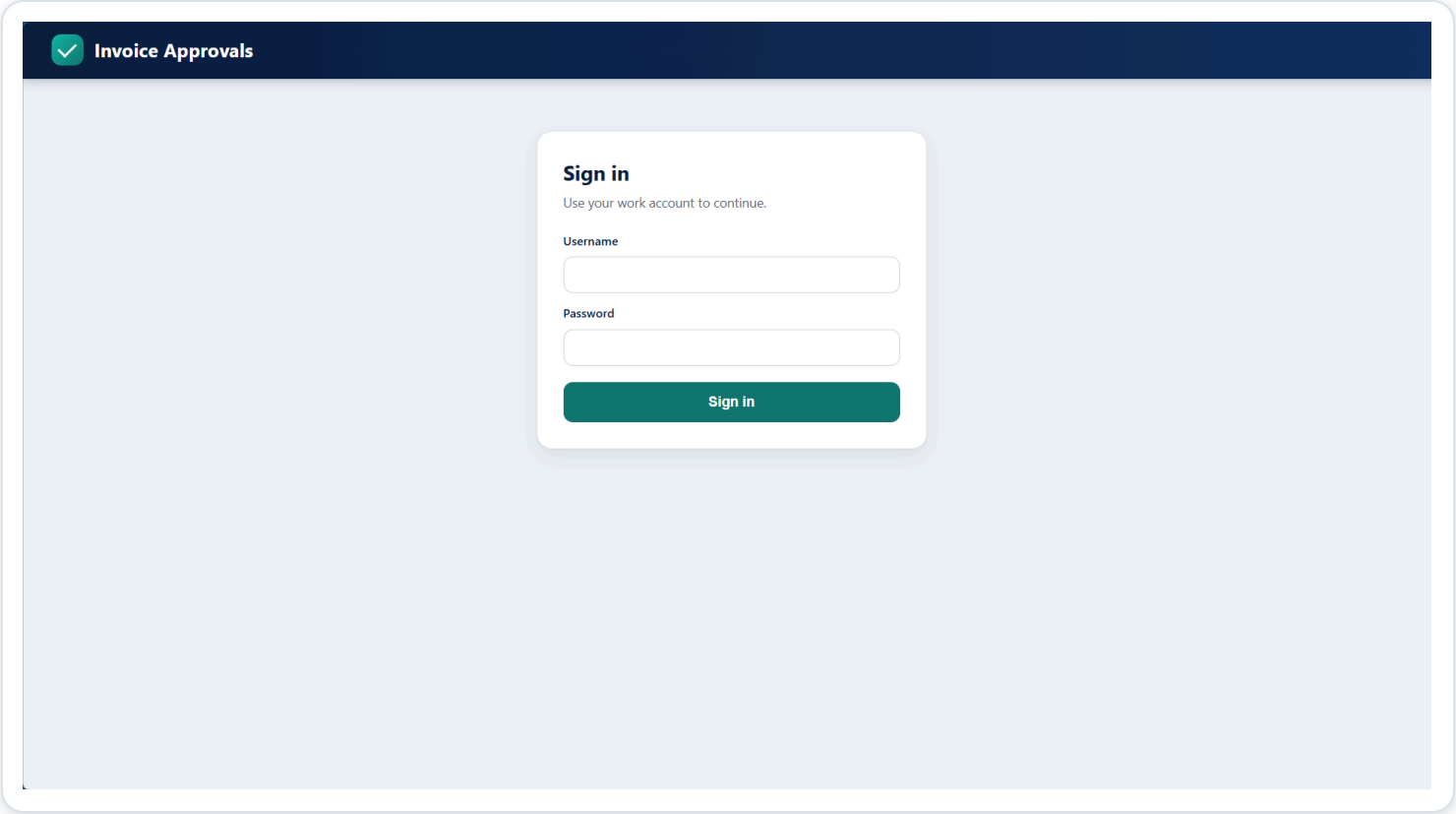
INV-2001 · 75,000.00

Needs one Director. A Manager is refused.

INV-2002 · 600,000.00

Needs two different approvers, the second a Director.

The sign-in screen



Not signed in

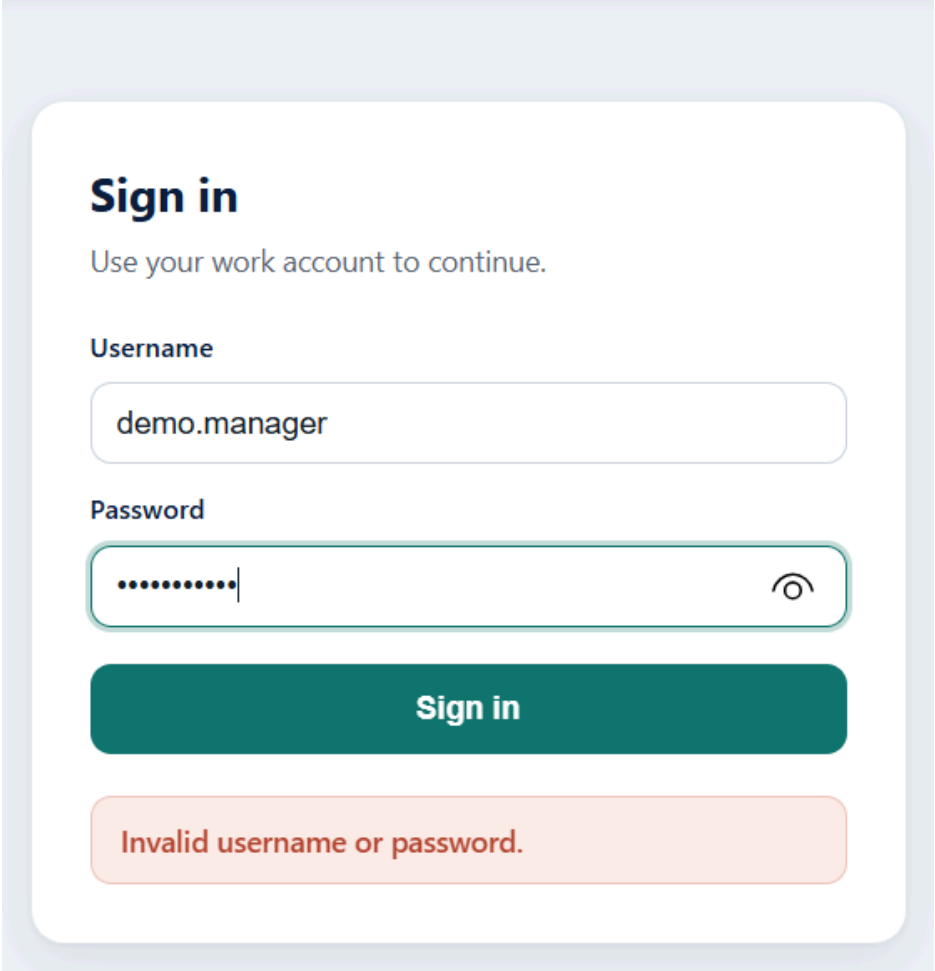
What you see

- Only a sign-in card is shown; there is **no Sign out button** and no invoice data.
- Every part of the app is behind sign-in: the server answers 401 to anything else.
- Username and password; one button.

What it proves

Nothing is visible before sign-in, and the page does not pretend you are signed in (a bug where Sign out showed here was found and fixed).

A wrong password is refused



Not signed in

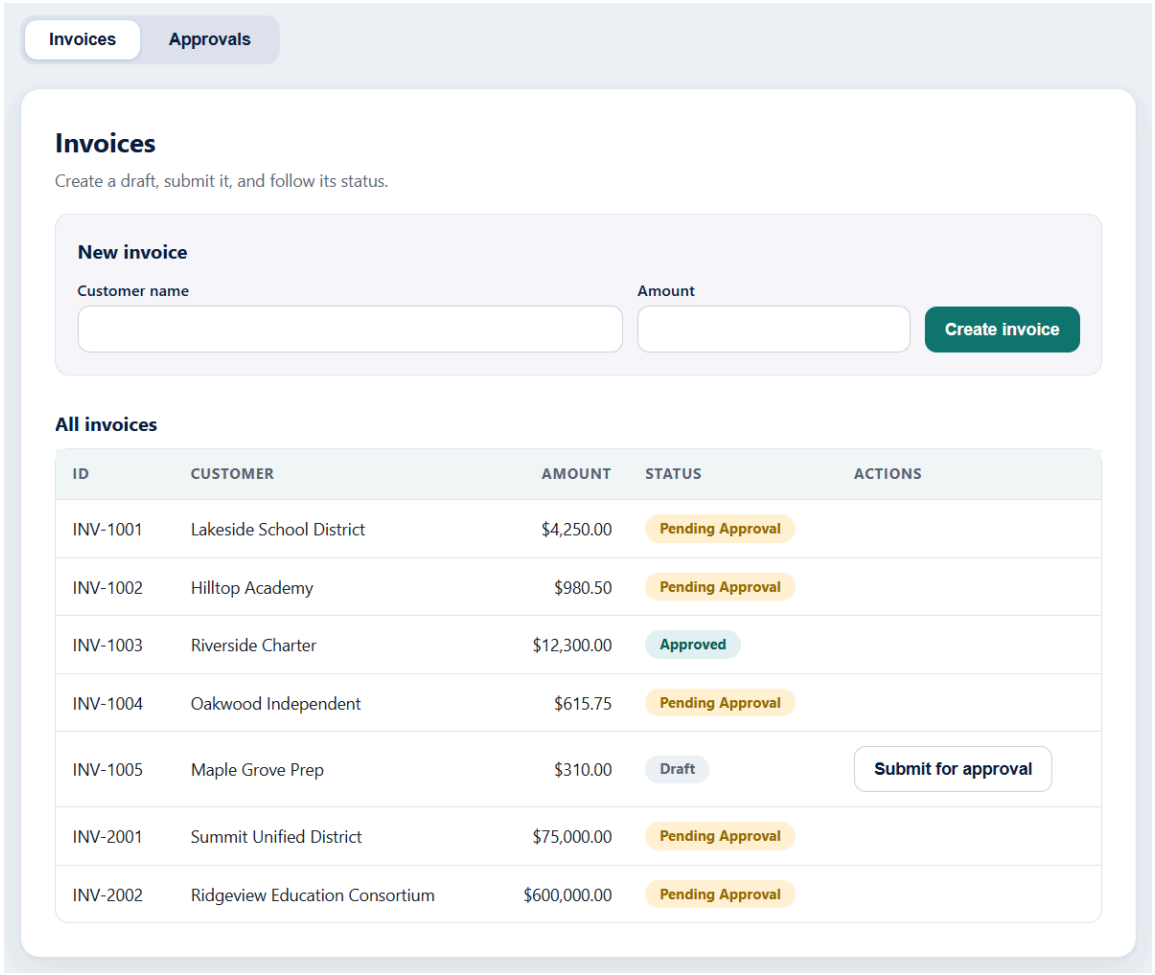
What you see

- The message **Invalid username or password.** appears in a red notice under the button.
- It does not say which of the two was wrong.
- The password field is masked.

What it proves

Sign-in failures are generic by design, so the form cannot be used to find out which usernames exist.

The Invoices tab



Signed in as demo.manager (Manager)

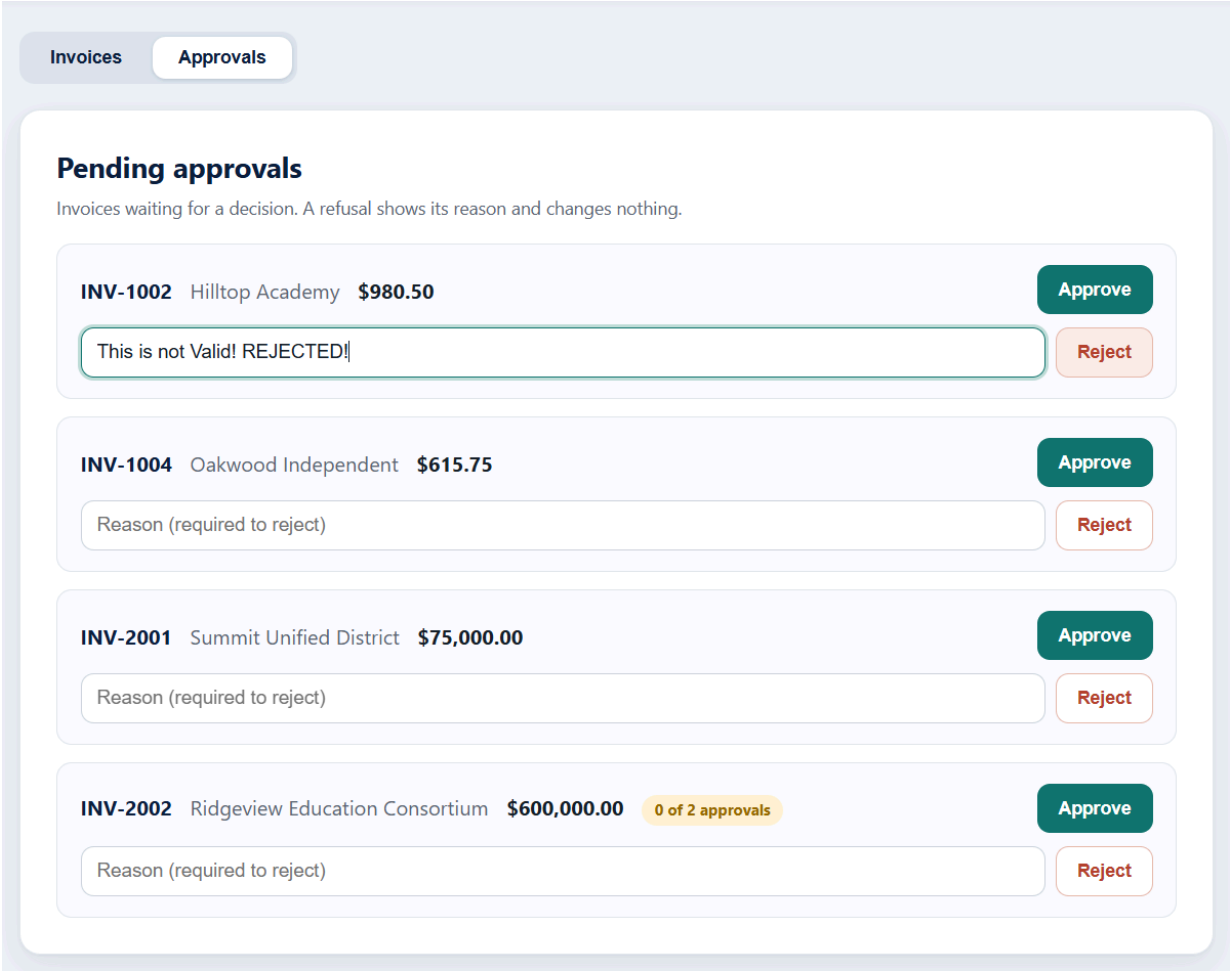
What you see

- A **New invoice** form (customer name, amount) and the **All invoices** table.
- Status pills: **Pending Approval** (amber), **Approved** (teal), **Draft** (grey), **Rejected** (red).
- A Draft invoice has a **Submit for approval** button.
- INV-2001 (75,000.00) and INV-2002 (600,000.00) are the invoices built to try the rules.

What it proves

The seed data covers every status, so each rule can be tried without creating anything first.

Rejecting needs a reason



Signed in as demo.manager (Manager)

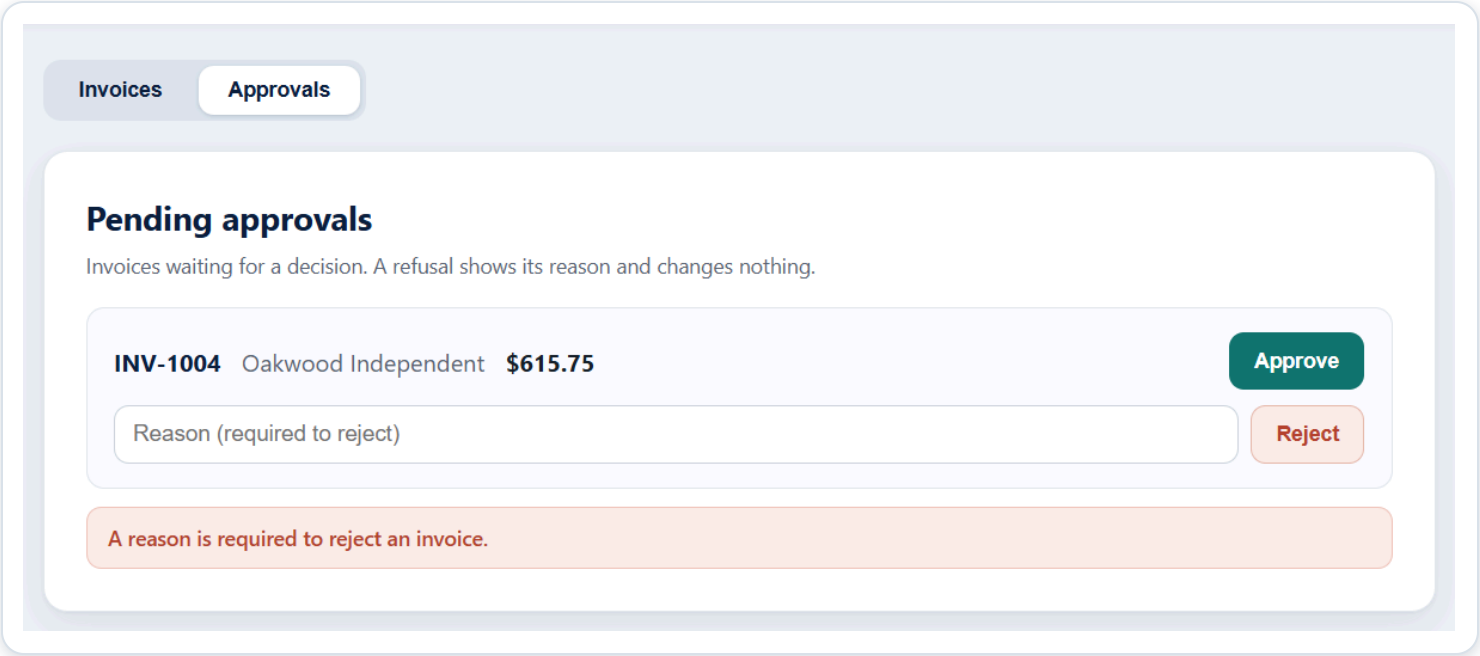
What you see

- The **Approvals** tab lists every invoice waiting for a decision.
- Each card has **Approve**, a reason box and **Reject**.
- A reason has been typed for INV-1002.
- INV-2002 shows **0 of 2 approvals**: it needs two people.

What it proves

Reject is a different action from a refused approval: it ends the invoice, and it requires a reason that is stored and shown later.

Reject without a reason is refused



Signed in as demo.director2 (Director)

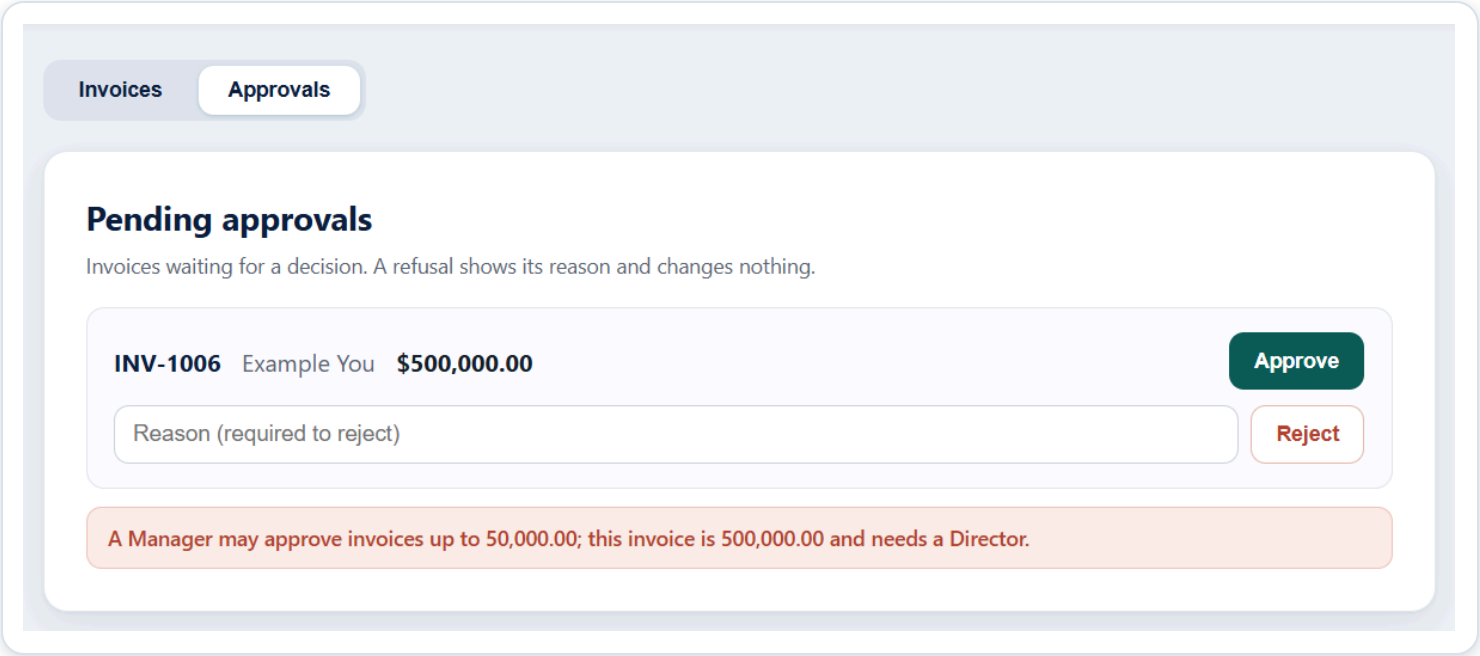
What you see

- Reject was pressed with the reason box empty.
- A red notice says **A reason is required to reject an invoice.**
- INV-1004 is still in the list: nothing was changed.

What it proves

A missing reason is bad input (HTTP 400). It is caught before the invoice is touched.

A Manager is refused above 50,000.00



Signed in as demo.manager (Manager)

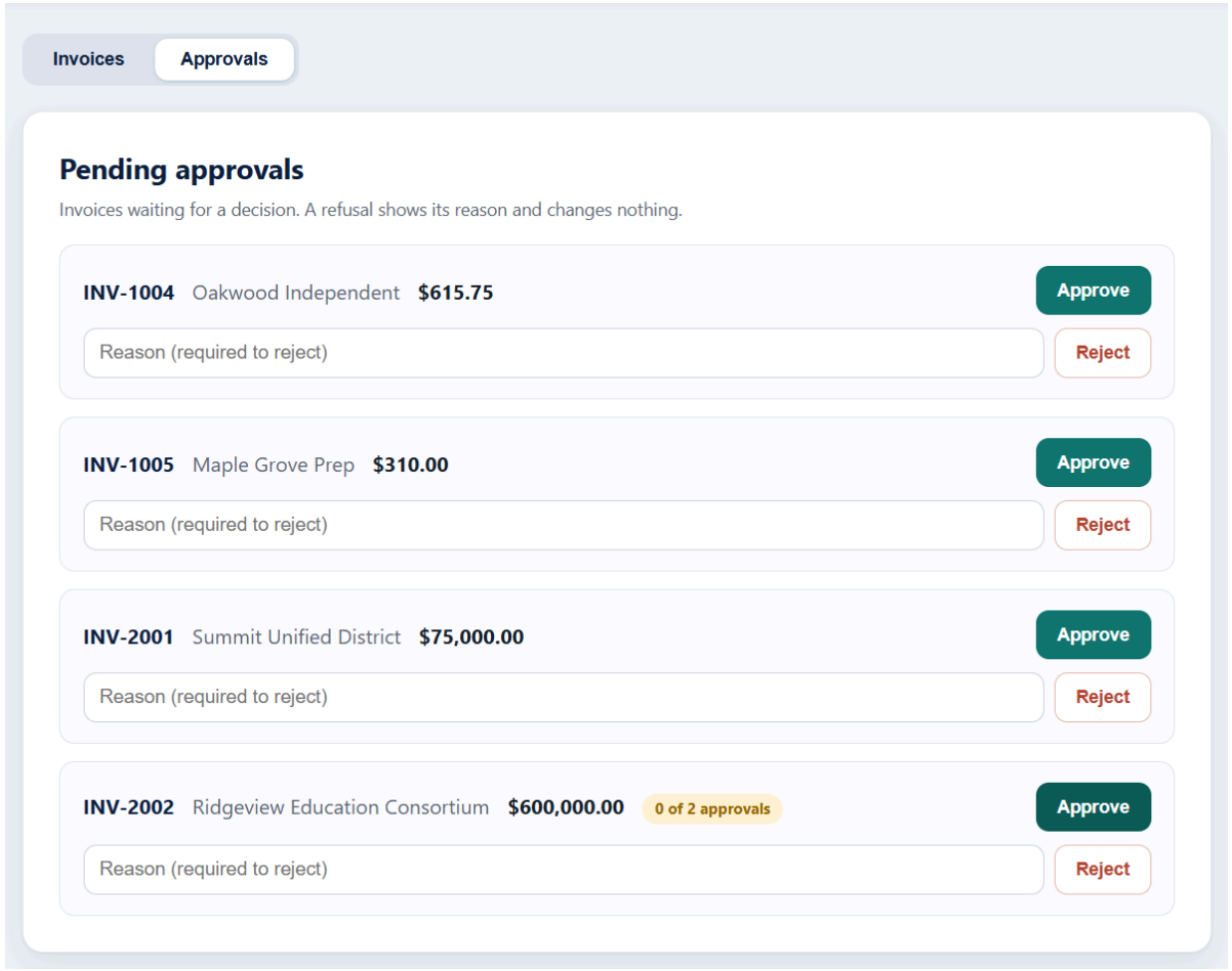
What you see

- Approve was pressed on INV-1006, **500,000.00**.
- The reason is shown in full: **A Manager may approve invoices up to 50,000.00; this invoice is 500,000.00 and needs a Director.**
- The invoice is still waiting; the Approve button is still there.

What it proves

The limit is enforced by role, the user is told exactly why, and a refusal changes nothing. This is the screen the browser test checks.

A Director sees the larger invoices



Signed in as demo.director (Director)

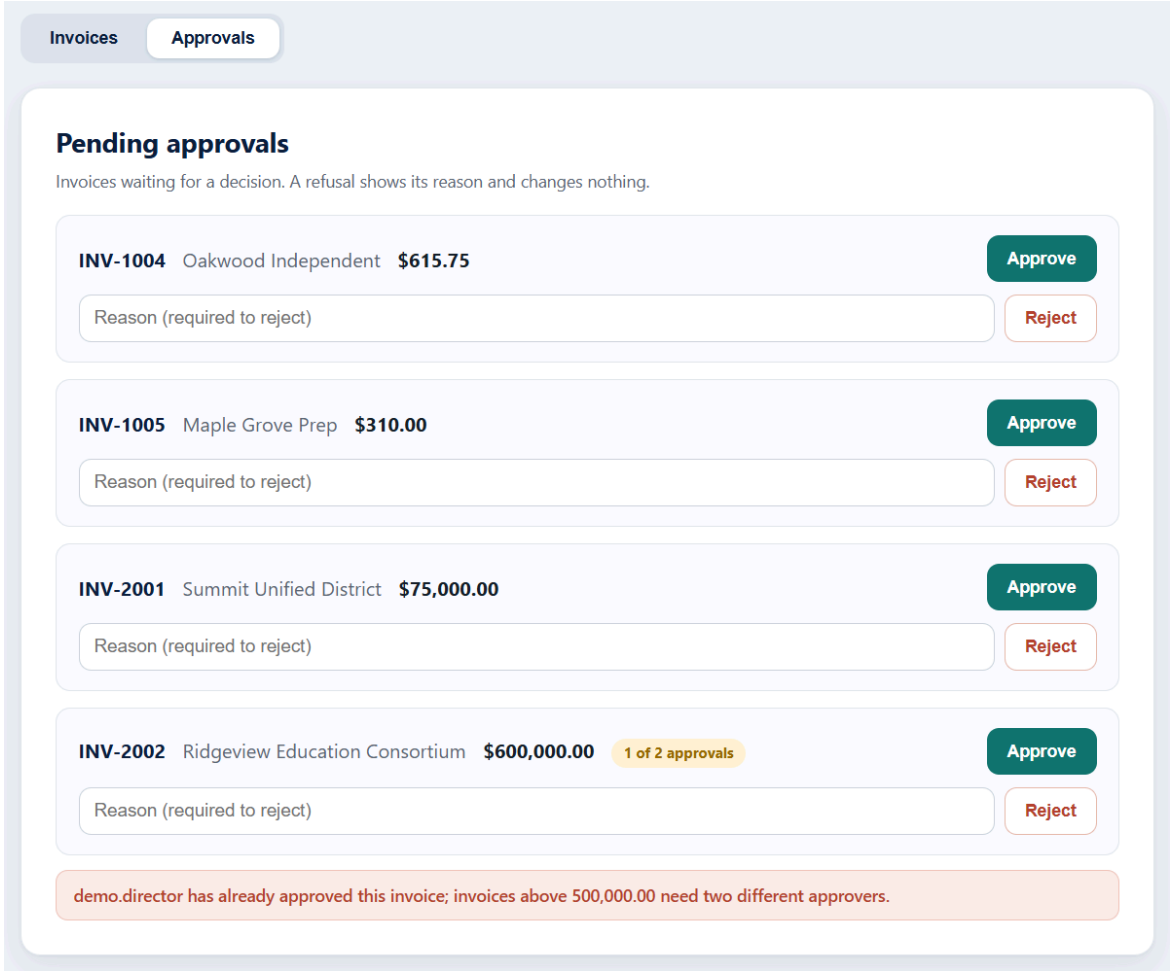
What you see

- The same list, signed in as a Director.
- INV-2001 (75,000.00) can be approved here; INV-2002 is still **0 of 2 approvals**.
- The header shows who is signed in and their role.

What it proves

Between 50,000.01 and 500,000.00 one Director is enough. Above 500,000.00 the first approval only moves the counter.

The same Director cannot approve twice



Signed in as demo.director (Director)

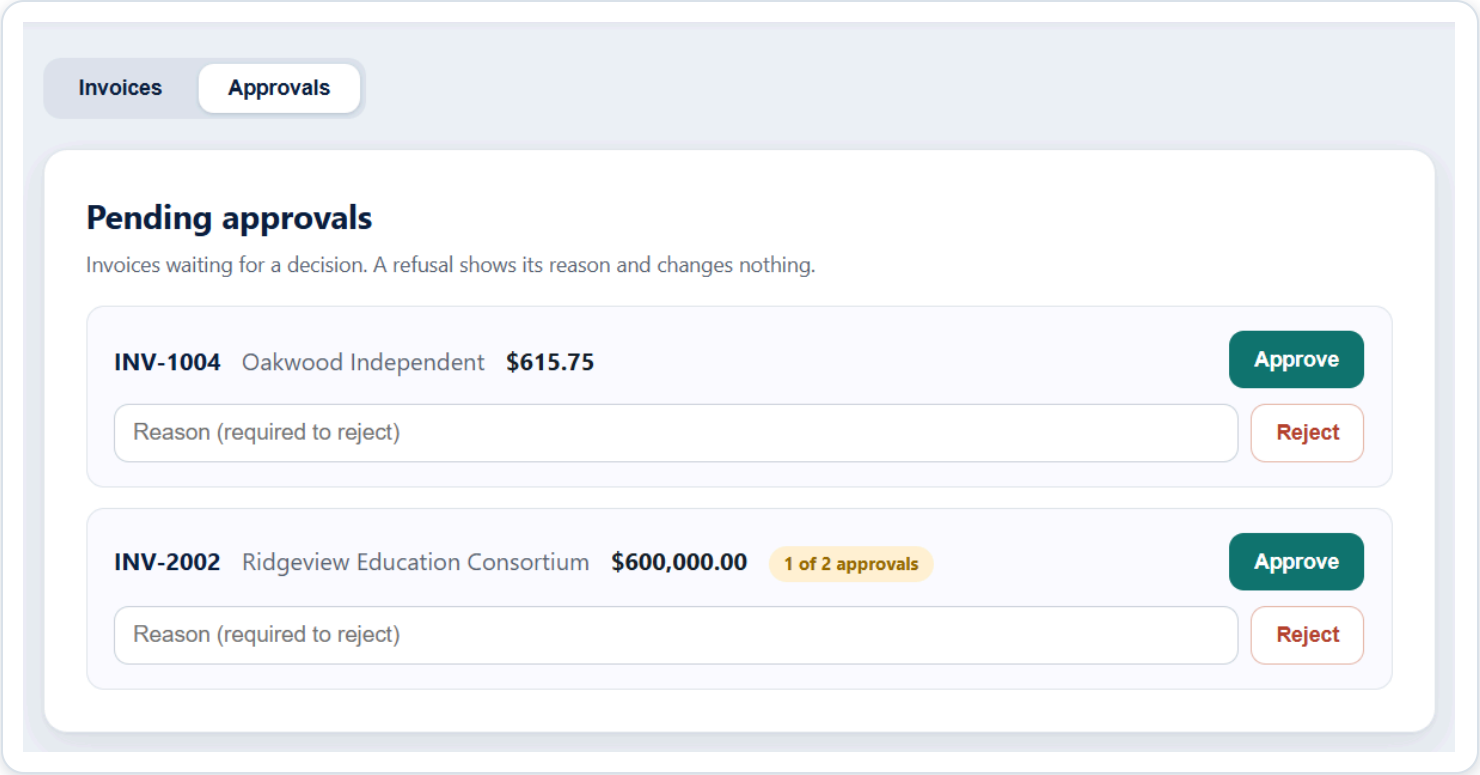
What you see

- INV-2002 now shows **1 of 2 approvals**: this Director approved once.
- Approving again gives: **demo.director has already approved this invoice; invoices above 500,000.00 need two different approvers.**
- INV-2001, already approved, has left the list.

What it proves

The first negative rule: the same user never counts as both approvers, and the refused second attempt changes nothing.

A second, different Director takes over



Signed in as demo.director2 (Director)

What you see

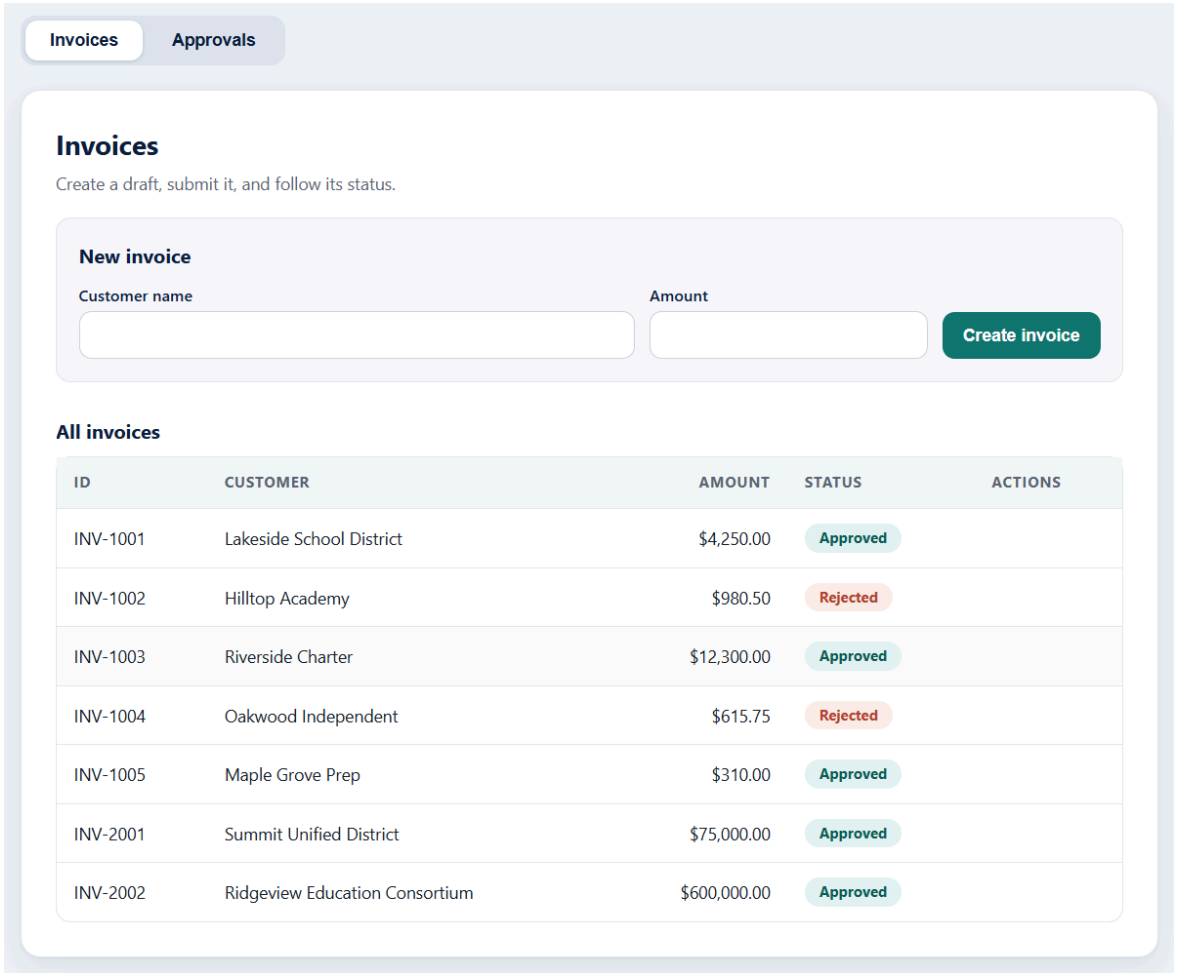
- After signing out and in as **demo.director2**, INV-2002 still reads **1 of 2 approvals**.
- The first approval was kept; it is waiting for a different Director.
- The message from the previous screen is gone: it is cleared on sign-out and sign-in.

What it proves

The invoice has been waiting in a valid state, not lost or half-written.

THE RULES

Two approvals complete the invoice



Signed in as demo.director2 (Director)

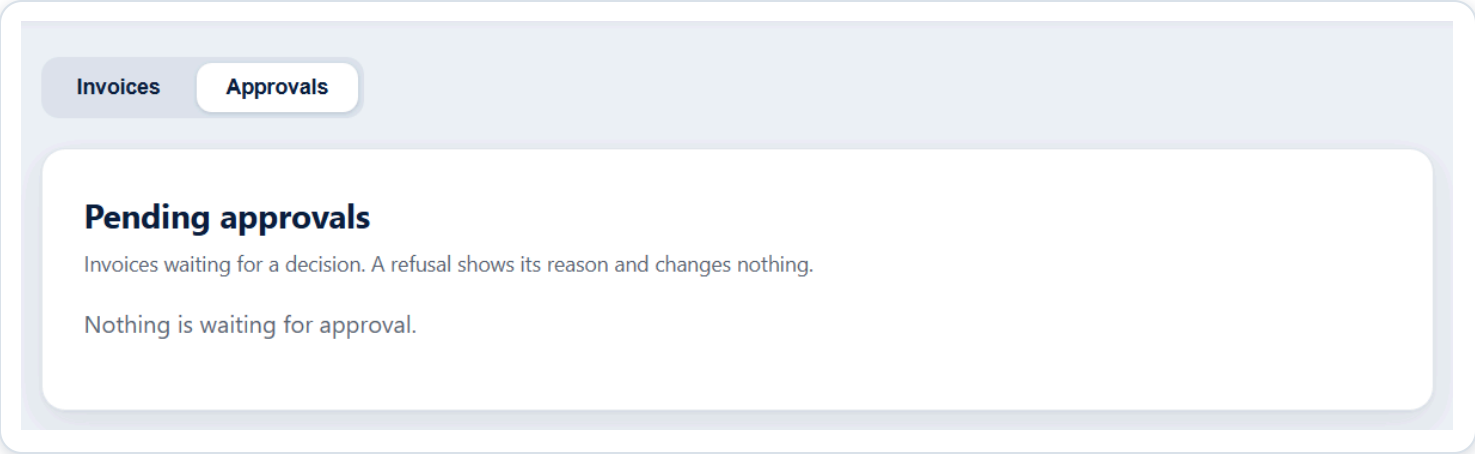
What you see

- On the Invoices tab INV-2002 is now **Approved**.
- Rejected invoices carry a red **Rejected** pill.
- Nothing is left in a half-approved state.

What it proves

Above 500,000.00 the invoice becomes Approved only when two different users, the second a Director, have approved.

Admin has nothing to approve



Signed in as demo.admin (Admin)

What you see

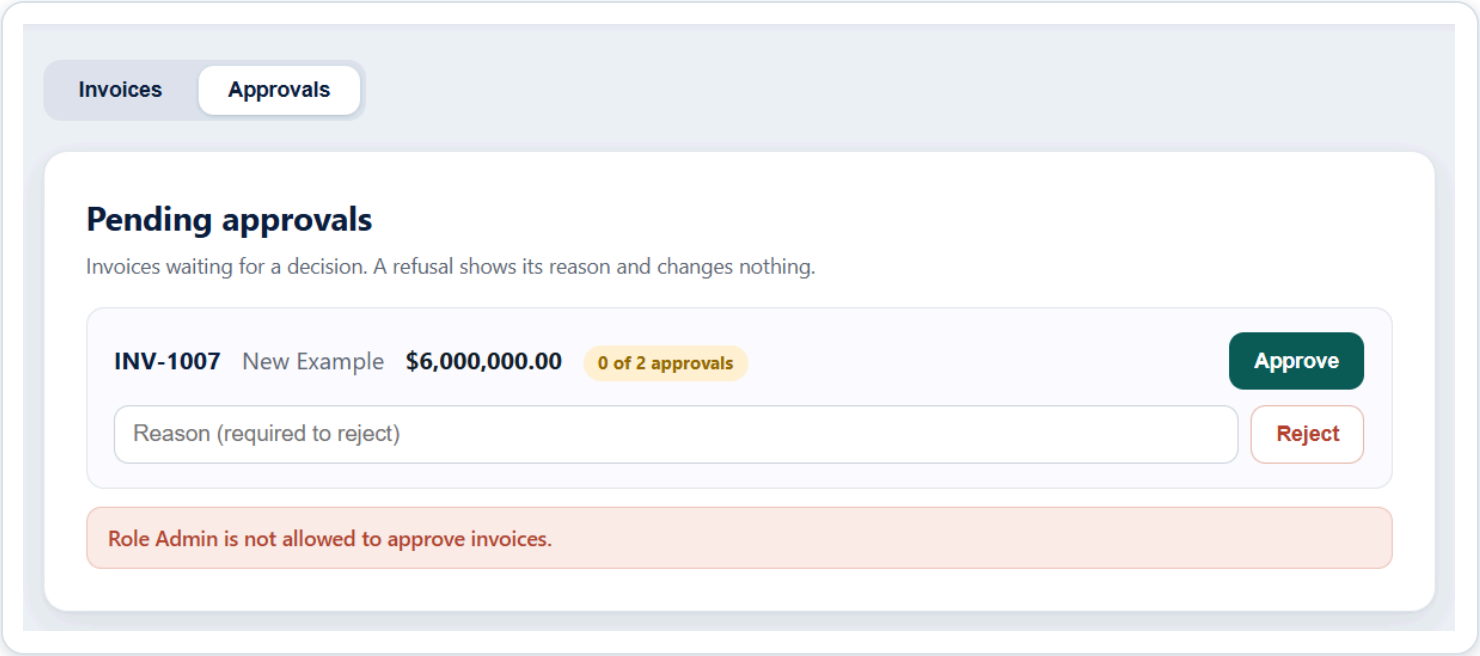
- The Approvals tab shows **Nothing is waiting for approval.** once every invoice has a decision.
- An empty list is a normal state with its own message.

What it proves

Every pending invoice went through the rules; the queue is empty and nothing is stuck.

THE RULES

Admin may look, but not approve



Signed in as demo.admin (Admin)

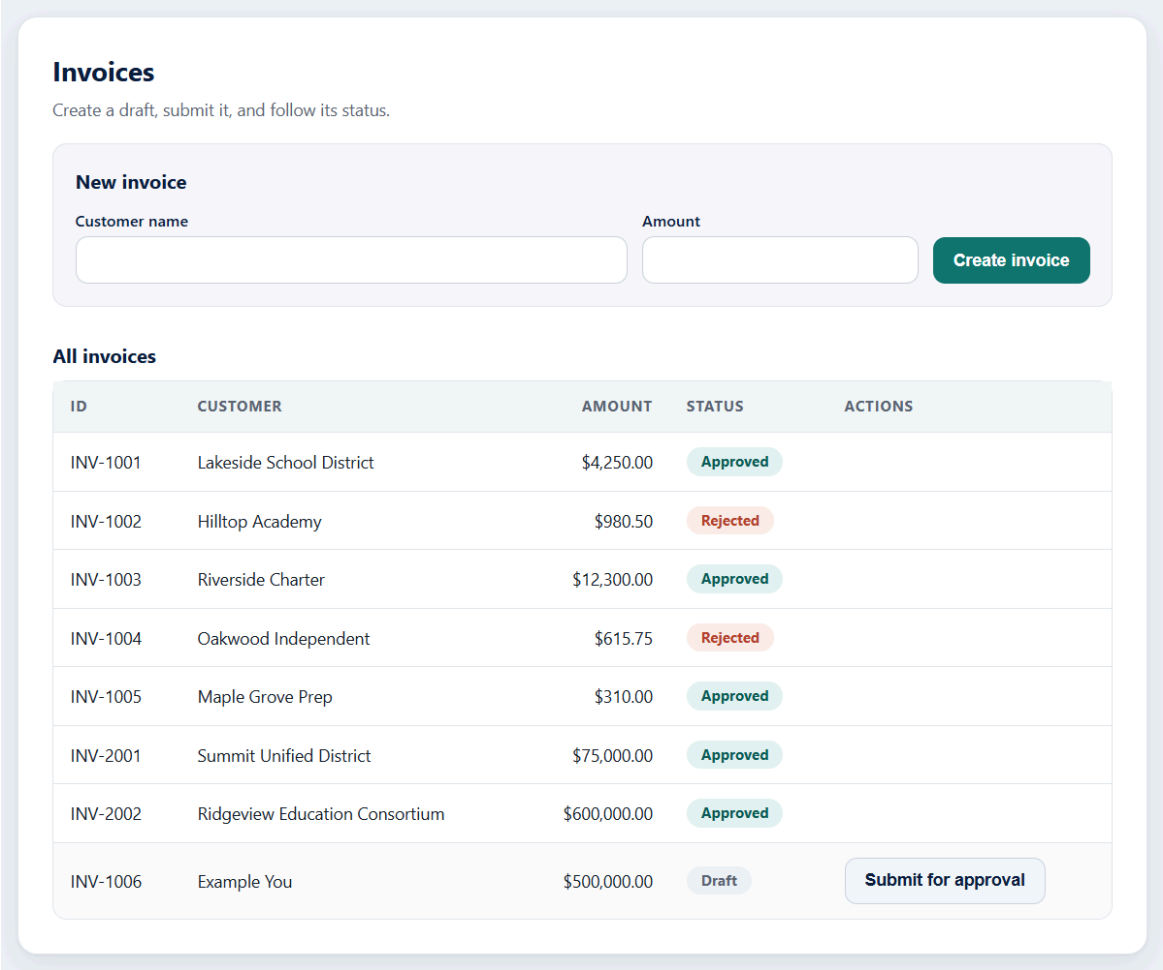
What you see

- INV-1007 (6,000,000.00) is waiting, **0 of 2 approvals**.
- Approve was pressed as Admin: **Role Admin is not allowed to approve invoices.**
- The invoice is unchanged.

What it proves

Admin can reject and delete but never approve. The role check comes first, before any amount rule.

Creating a draft invoice



Signed in as demo.admin (Admin)

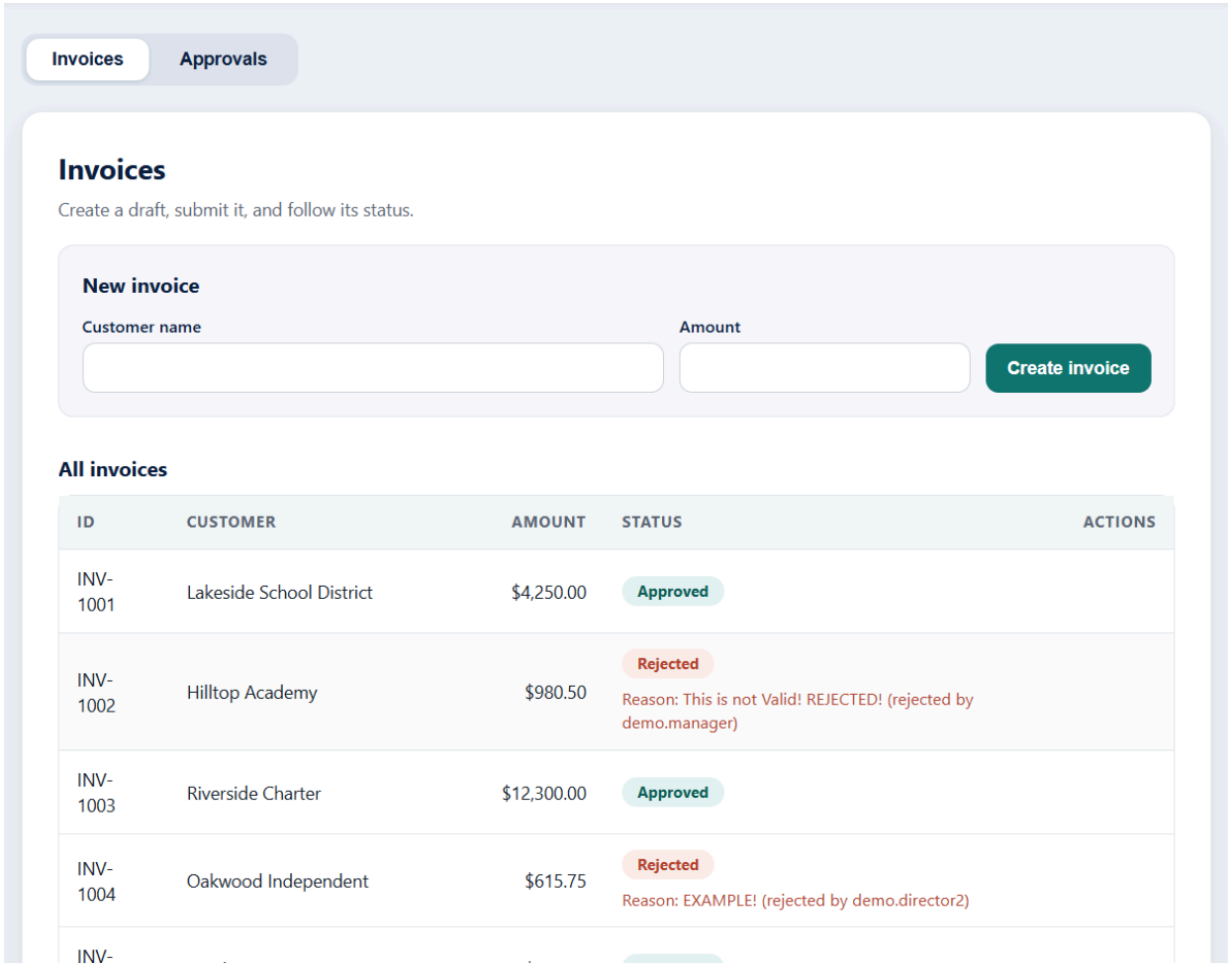
What you see

- A new invoice **INV-1006** for 500,000.00 appears as **Draft**.
- It has a **Submit for approval** button; it is not in anyone's approval queue yet.
- Amounts are checked on the server: zero, negative, sub-cent and huge values are rejected.

What it proves

An invoice moves Draft, then Pending, then Approved or Rejected. Amounts are exact decimals, never floating point.

Rejection reasons stay visible



Signed in as demo.director (Director)

What you see

- Rejected invoices show the reason under the pill: **Reason: … (rejected by demo.manager)**.
- Anyone who opens the Invoices tab can see who rejected an invoice and why.

What it proves

A rejection is accountable: the decider and the reason are stored with the invoice and shown to everyone.

EVIDENCE

Unit tests: 177 passed

```
python -m pytest backend/tests -q
```

```
177 passed, 1 warning in 1.23s
```

What ran

63 policy tests, 62 service tests, 36 API tests and 16 repository-contract tests.

What they prove

Every rule and boundary, the two negative rules, and that a refusal leaves the invoice unchanged.

One warning

A third-party deprecation notice about the test client; it does not affect the result.

EVIDENCE

The two negative rules, run on their own

```
python -m pytest backend/tests/test_approval_policy.py -v -k "same_user or second_approver"
```

```
collected 63 items / 51 deselected / 12 selected

backend\tests\test_approval_policy.py::test_second_approver_must_be_a_different_director PASSED                                  [  8%]
backend\tests\test_approval_policy.py::test_second_approver_who_is_a_manager_is_refused_with_a_reason PASSED                     [ 16%]
backend\tests\test_approval_policy.py::test_same_user_cannot_be_both_approvers PASSED                                            [ 25%]
backend\tests\test_approval_policy.py::test_same_user_is_recognised_despite_case_and_whitespace[DIR.ONE] PASSED                  [ 33%]
backend\tests\test_approval_policy.py::test_same_user_is_recognised_despite_case_and_whitespace[ dir.one ] PASSED               [ 41%]
backend\tests\test_approval_policy.py::test_same_user_is_recognised_despite_case_and_whitespace[Dir.One] PASSED                  [ 50%]
backend\tests\test_approval_policy.py::test_same_user_is_recognised_despite_case_and_whitespace[dir.one\t] PASSED               [ 58%]
backend\tests\test_approval_policy.py::test_same_user_check_wins_over_the_role_check PASSED                                     [ 66%]
backend\tests\test_approval_policy.py::test_is_same_user_ignores_case_and_surrounding_whitespace_only PASSED                    [ 75%]
backend\tests\test_approval_policy.py::test_same_user_is_recognised_when_the_stored_name_has_odd_case_or_spacing[ Dir.One ] PASSED [ 83%]
backend\tests\test_approval_policy.py::test_same_user_is_recognised_when_the_stored_name_has_odd_case_or_spacing[DIR.ONE] PASSED [ 91%]
backend\tests\test_approval_policy.py::test_same_user_is_recognised_when_the_stored_name_has_odd_case_or_spacing[dir.one\t] PASSED [100%]

====================================================== 12 passed, 51 deselected in 0.04s ======================================================
```

Same user twice

Refused, including `Dir.One`, `dir.one` and tab or space padding.

Manager as second approver

Refused with a reason above 500,000.00.

12 passed

51 other tests were skipped by the filter; this view is only the two rules.

EVIDENCE

Mutation testing: 100%

```
wsl -d Ubuntu-22.04 -- bash scripts/run_mutation.sh demo
```

```
Mutation report: demo
Tool: mutmut mutmut, version 3.8.0
Scope: src/invoice_app/domain (tests: tests/test_approval_service.py tests/test_approval_policy.py if present)

Total mutants : 132
Killed        : 132
Timeout       : 0
Survived      : 0
Other         : 0
Mutation score: 100.0% (killed+timeout / total)

Survivors:
```

What it does

Changes the domain code 132 different ways, one at a time, and runs the tests.

The result

132 killed, 0 survived: the tests noticed every change.

Why it matters

A high line-coverage number does not say whether tests can fail; this does.

EVIDENCE

Browser test: 1 passed

```
cd e2e; npx playwright test --headed; cd ..
```

```
Running 1 test using 1 worker

  ✓  1 …hromium] › tests\over-limit-rejection.spec.ts:14:5 › a manager is refused on an over-limit invoice, sees why, and the invoice does not change (1.4s)

  1 passed (3.1s)

To open last HTML report run:

  npx playwright show-report
```

What it does

Drives the real UI as a Manager and tries to approve an over-limit invoice.

What it checks

The exact refusal reason, and that the invoice stays Pending Approval.

Clean up

The test creates its own invoice and removes it, so it can run again and again.

SUMMARY

Every screen, and the rule it proves

Screen	Rule or behaviour proven
Sign-in and wrong password	Everything is behind sign-in; failures are generic.
Reject with and without a reason	A reason is required, stored and shown with who rejected.
Manager refused on 500,000.00	Role limits are enforced with a clear reason; nothing changes.
Same Director refused on INV-2002	The same user never counts twice above 500,000.00.
Second Director completes INV-2002	Two different approvers, the second a Director, are required.
Admin refused	Admin can reject and delete but never approve.
Draft and Submit	Invoices move Draft, Pending, then Approved or Rejected.
Test, mutation and browser output	177 tests, 100% mutation score and 1 browser test, all repeatable.

Try it yourself

- Start the app: `python -m uvicorn invoice_app.api.main:app --app-dir backend/src --port 5000`
- Open `http://localhost:5000` and sign in as `demo.manager`
- Approve INV-2001 and read the refusal, then try again as `demo.director`
- Approve INV-2002 as `demo.director`, then as `demo.director2`